

LIBRARY MANAGEMENT SYSTEM

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the
award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

Y. RENUKA LAHARI – 25B11AIC88

CH. LAKSHMI NARASIMHA – 25B11AI231

K. SARANYA – 25B11AI458

CH. HARISWAR—25B11AI227

Under the esteemed guidance of

Mr. T.Srinivasulu M.Tech, (Ph.D.)

Assistant Professor



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE
LEARNING**

ADITYA UNIVERSITY

(Formerly Aditya Engineering College (A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2026 – 2027

ADITYA UNIVERSITY

(Formerly Aditya Engineering College(A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled

“LIBRARY MANAGEMENT SYSTEM” is being submitted by Y. RENUKA LAHARI (Roll No. 25B11AIC88), CH. LAKSHMI NARASIMHA (Roll No. 25B11AI231), K. SARANYA (Roll No. 25B11AI458) and CH. HARISWAR (Roll No. 25B11AI227) in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the **Capstone Project** for the course **Database Management Systems (2501IT05)** during the academic year **2026–2027**, and records the database design, implementation, and documentation undertaken by the team.

Project Guide

Mr. T. Srinivasulu, M. Tech, (Ph.D.)
Associate professor
HOD

Head of the Department

Dr. K. Naga Bhargavi, M. Tech, Ph.D.
Associate Professor &

External Examiner

DECLARATION

We hereby declare that the DBMS Capstone Project entitled “**Library Management System**” is a genuine work carried out by us. The database design, SQL implementation, queries, testing, and documentation presented in this report are our own work, completed under the guidance of **Mr. T.Srinivasulu**, Department of **Artificial Intelligence and Machine Learning**, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates

Y. RENUKA LAHARI (25B11AIC88)

CH. LAKSHMI NAKASHIMA (25B11AI231)

K. SARANYA(25B11AI458)

CH. HARASIWAR(25B11AI227)

Date: _____

Place: Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide **Mr. T.Srinivasulu**, Department of **Artificial Intelligence and Machine Learning**, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to **K NagaBhargavi**, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

Project Associates

Y. RENUKA LAHARI -- 25B11AIC88

CH. LAKSHMI NAKASHIMA -- 25B11AI231

K. SARANYA -- 25B11AI458

CH. HARASIWAR -- 25B11AI227

INDEX

Chapters	Title	Page No.
	Certificate	
	Declaration	
	Acknowledgement	
	Abstract	
	List of Figures	
	List of Tables	
1	Introduction	
	1.1 Introduction to the Domain	
	1.2 Problem Statement	
	1.3 Motivation and Need for a Database	
	1.4 Objectives of the Project	
	1.5 Scope of the Project	
	1.6 Organization of the Report	
2	System Study and Tools	
	2.1 Existing System and Its Limitations	
	2.2 Proposed Database System	
	2.3 Tools and Technologies Used	
	2.4 Software and Hardware Requirements	
3	Database Design (Review – I)	
	3.1 Identified Entities	
	3.2 Entity Attributes and Attribute Types	
	3.3 ER Diagram	
	3.4 Relationships and Cardinality	
	3.5 Relational Database Schema	
	3.6 Primary Keys and Foreign Keys	
	3.7 Normalization up to 3NF	
4	Database Implementation (Review – II)	
	4.1 Database and Table Creation – DDL	
	4.2 Sample Data Insertion – DML	
	4.3 Constraints and Data Integrity	
	4.4 Data Validation Rules	
5	SQL Operations and Queries	
	5.1 DQL – Basic Retrieval Queries	
	5.2 Join Operations	
	5.3 Aggregate Functions, GROUP BY and HAVING	
	5.4 Nested and Correlated Subqueries	
	5.5 Views and Relational Set Operations	
	5.6 DCL and TCL Operations	

6	Modules and Functionalities	
7	Results – Sample Outputs and Screenshots	
8	Testing and Data Validation	
9	GitHub Repository and Team Contribution	
10	Conclusion and Future Enhancements	
	References	
	Appendix A – Complete SQL Script	



LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
Figure 1.1	ER Diagram of Library Management System	
Figure 1.2	Relational Database Schema	
Figure 1.3	Book Table Structure	
Figure 1.4	Author Table Structure	
Figure 1.5	Publisher Table Structure	
Figure 1.6	Member Table Structure	
Figure 1.7	Loan Table Structure	
Figure 1.8	Fine Table Structure	
Figure 1.9	Sample Book Records	
Figure 1.10	Sample Loan Records	
Figure 1.11	SQL Query Execution	
Figure 1.12	SQL Query Output	
Figure 1.13	Database Constraints Implementation	
Figure 1.14	Normalization from 1NF to 3NF	

LIST OF TABLES

Table No.	Title of the Table	Page No.
Table 1.1	Book Entity and Attributes	
Table 1.2	Author Entity and Attributes	
Table 1.3	Publisher Entity and Attributes	
Table 1.4	Member Entity and Attributes	
Table 1.5	Loan Entity and Attributes	
Table 1.6	Fine Entity and Attributes	
Table 1.7	Relationships Between Entities	
Table 1.8	Book Sample Data	
Table 1.9	Loan Sample Data	
Table 1.10	SQL DDL Operations	
Table 1.11	SQL DML Operations	
Table 1.12	SQL Queries and Their Purpose	
Table 1.13	Constraints and Data Integrity Rules	
Table 1.14	Normalization: 1NF to 3NF	

CHAPTER 1: INTRODUCTION

1.1 Introduction to the Domain

A library is an important organisation that provides users with access to books and other learning resources. A library regularly manages information about books, authors, publishers, members, book loans, and fines. The Library Management System focuses on organizing these activities in a structured manner so that library records can be stored and managed efficiently.

The system maintains details such as book titles, ISBN numbers, total copies, genres, publication information, author details, publisher information, and member records. It also records loan information, including the book borrowed, member who borrowed it, loan date, due date, return date, and loan status. Fine information is maintained for applicable loans, including the fine amount, fine The database provides a systematic way to manage these records using relationships between the major entities: **Book, Author, Publisher, Member, Loan, and Fine**. It helps organize library information, maintain relationships between records, reduce data redundancy through normalization, and support efficient SQL-based data management.

1.2 Problem Statement

1. Manual Data Handling

- Library records may be maintained using registers, spreadsheets, or separate records.
- Managing large amounts of data manually is time-consuming.

2. Data Redundancy

- The same information may be stored in multiple places.
- This causes unnecessary duplication.

3. Data Inconsistency

- If information is updated in one place but not another, records can become inconsistent.
- Example: member or book details may differ between records.

4. Difficulty in Data Retrieval

- Finding book availability, borrower details, due dates, or fine information can require significant manual effort.

5. Lack of Data Integrity

- Without proper validation, duplicate or incorrect records may be entered.

- Relationships between different records are difficult to maintain.

6. Proposed Database Solution

- Store information in a centralized relational database.
- Use separate tables such as Book, Author, Publisher, Member, Loan, and Fine.

7. Integrity Constraints

- Primary Keys → unique identification.
- Foreign Keys → maintain relationships.
- UNIQUE → prevent duplicate values.
- NOT NULL → ensure required data is entered.
- CHECK → validate values.

1.3 Motivation and Need for a Database

A DBMS is required to manage library information such as books, authors, publishers, members, loans, and fines in an organized manner. Compared with files or spreadsheets, a DBMS provides faster data retrieval, easier updating, and better data organization. It also helps maintain data integrity, avoid data redundancy and inconsistency, and manage relationships between different tables using primary keys and foreign keys. Normalization up to 3NF further improves the structure of the database and reduces unnecessary duplication.

1.4 Objectives of the Project

1. To design a normalized relational database for the selected domain with at least six entities.
2. To implement the database with appropriate primary keys, foreign keys and integrity constraints.
3. To efficiently manage information about **books, authors, publishers, members, loans, and fines**
4. To provide **easy storage, retrieval, updating, and management** of library records using SQL queries.
5. To reduce **data redundancy and inconsistency** by applying normalization up to **Third Normal Form (3NF)**.

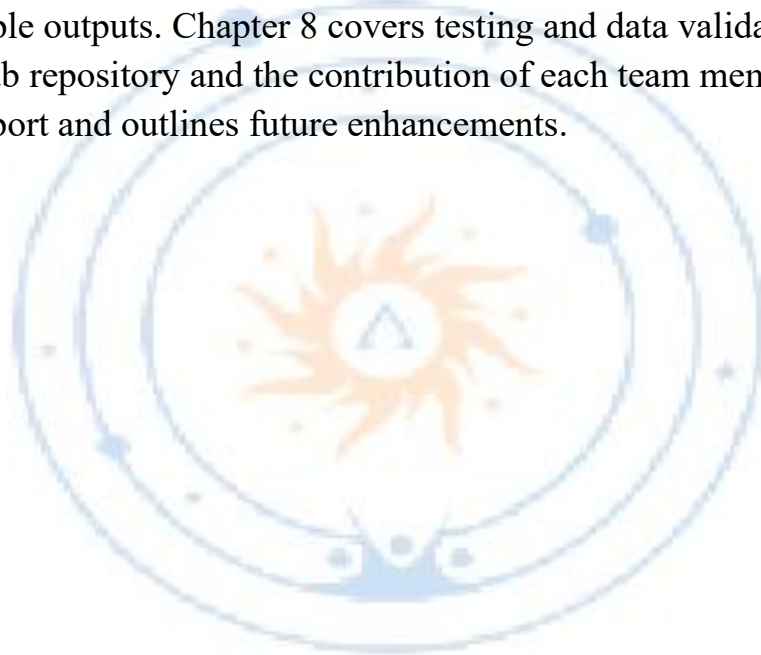
1.5 Scope of the Project

The Library Management System covers the management of **books, authors, publishers, members, loans, and fines**. It supports storing, updating, and retrieving library records using SQL. The database also manages relationships between entities and maintains data integrity using keys and constraints.

The project **does not cover** advanced features such as online book purchasing, online payment processing, automated notifications, or integration with external library systems. The focus is on demonstrating the **database design, relationships, SQL operations, normalization, and data integrity** of a Library Management System.

1.6 Organization of the Report

This report is organized into ten chapters. Chapter 2 presents the system study and the tools used. Chapter 3 covers the complete database design including the ER diagram, relational schema and normalization. Chapter 4 describes the database implementation with DDL and sample data. Chapter 5 presents the SQL operations and queries executed. Chapter 6 explains the modules and functionalities. Chapter 7 presents the results with sample outputs. Chapter 8 covers testing and data validation. Chapter 9 details the GitHub repository and the contribution of each team member. Chapter 10 concludes the report and outlines future enhancements.



CHAPTER 2: SYSTEM STUDY AND TOOLS

2.1 Existing System and Its Limitations

In the existing system, library information may be managed using **manual registers, files, or spreadsheets**. Details such as books, authors, publishers, members, loans, returns, and fines are maintained separately. As the amount of information increases, maintaining and updating these records becomes difficult.

Limitations of the Existing System:

- **Data redundancy** – the same information may be stored multiple times.
- **Data inconsistency** – updated information may not be reflected in all records.
- **Difficult data retrieval** – finding book, member, loan, or fine details takes more time.
- **Lack of data integrity** – there may be no proper validation of records.
- **Difficulty in maintaining relationships** – connections between books, authors, members, loans, and fines are harder to manage.
- **Higher risk of errors** – manual data entry can result in incorrect or missing information.
- **Limited security and control** – files and spreadsheets provide less structured control over data access.

2.2 Proposed Database System

The proposed **Library Management System** uses a structured relational database to manage information about **books, authors, publishers, members, loans, and fines** in an organized manner. The system stores data in separate but related tables and uses **primary keys and foreign keys** to maintain relationships between records. Constraints such as **UNIQUE, NOT NULL, and CHECK** are used to maintain data integrity and accuracy. The system provides **faster and easier data retrieval, updating, and management** through SQL queries, while reducing data redundancy and inconsistency. **Normalization up to Third Normal Form (3NF)** is applied to improve database organization and eliminate unnecessary duplication. Overall, the proposed system provides a structured, reliable, and efficient way to manage library records compared with manual registers, files, or spreadsheets.

2.3 Tools and Technologies Used

Table 2.1: Tools and Technologies Used

Category	Tool / Technology Used	Version / Remarks
Database Server	PostgreSQL	Relational Database Management System
Query / Admin Tool	dbfiddle.dev	PostgreSQL environment
SQL	SQL	DDL, DML, DQL and constraints
ER Diagram Tool	draw.io (diagrams.net)	ER diagram design
Programming / Scripting	SQL	Database implementation and queries
Version Control	GitHub	Project repository and collaboration
Documentation	MS Word	Project report preparation
Presentation	Microsoft PowerPoint	Project presentation
Operating System	Windows	Development environment

2.4 Software and Hardware Requirements

Table 2.2: Software and Hardware Requirements

Requirement Type	Specification
Operating System	Windows 10 / Windows 11
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB
Storage	Minimum 10 GB free space
Database Server	PostgreSQL
Database Tool	dbfiddle.dev
Web Browser	Google Chrome / Microsoft Edge
ER Diagram Tool	draw.io (diagrams.net)
Version Control	GitHub
Documentation Tool	Microsoft Word
Presentation Tool	Microsoft PowerPoint
Internet Connection	Required for dbfiddle.dev, GitHub, and online tools

CHAPTER 3: DATABASE DESIGN

(Deliverables of Review – I)

3.1 Identified Entities

Table 3.1: Entities Identified for the Selected Domain

S.No	Entity Name	Description
1	Book	Stores information about books available in the library, including title, ISBN, genre, and number of copies.
2	Author	Stores details of authors who have written the books in the library.
3	Publisher	Stores information about publishers responsible for publishing the books.
4	Member	Stores details of library members who can borrow books.
5	Loan	Records book borrowing and return details for library members.
6	Fine	Stores information about fines charged to members for overdue loans.

3.2 Entity Attributes and Attribute Types

Table 3.2: Attributes of Each Entity

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Book	book_id	INT	PK	Simple
	title	VARCHAR(200)	—	Simple
	isbn	VARCHAR(20)	—	Simple
	total_copies	INT	—	Simple
	available_copies	INT	—	Derived
	genre	VARCHAR(50)	—	Simple
	publication_year	INT	—	Simple
	publisher_id	INT	FK	Simple
Author	author_id	INT	PK	Simple
	first_name	VARCHAR(50)	—	Composite
	last_name	VARCHAR(50)	—	Composite
	nationality	VARCHAR(50)	—	Simple
Publisher	publisher_id	INT	PK	Simple
	publisher_name	VARCHAR(100)	—	Simple
	country	VARCHAR(50)	—	Simple
	website	VARCHAR(200)	—	Simple
Member	member_id	INT	PK	Simple

	first_name	VARCHAR(50)	—	Composite
	last_name	VARCHAR(50)	—	Composite
	email	VARCHAR(100)	—	Simple
	membership_type	VARCHAR(30)	—	Simple
	membership_date	DATE	—	Simple
	membership_expiry	DATE	—	Simple
Member_Phone	member_id	INT	PK, FK	Simple
	phone_number	VARCHAR(15)	PK	Multi-valued
Loan	loan_id	INT	PK	Simple
	book_id	INT	FK	Simple
	member_id	INT	FK	Simple
	loan_date	DATE	—	Simple
	due_date	DATE	—	Simple
	return_date	DATE	—	Simple
	days_overdue	INT	—	Derived
	status	VARCHAR(20)	—	Simple
Fine	fine_id	INT	PK	Simple
	loan_id	INT	FK	Simple
	member_id	INT	FK	Simple
	fine_amount	DECIMAL(8,2)	—	Simple
	fine_date	DATE	—	Simple
	payment_status	VARCHAR(20)	—	Simple
Book_Author	book_id	INT	PK, FK	Simple
	author_id	INT	PK, FK	Simple

3.3 ER Diagram

The ER diagram represents the **Library Management System** entities, attributes, relationships, and cardinalities. It includes **Book, Author, Publisher, Member, Loan, Fine, Book_Author, and**

Member_Phone. Primary keys and foreign keys define the relationships, while **1:N** and **M:N** cardinalities show how the entities are connected.

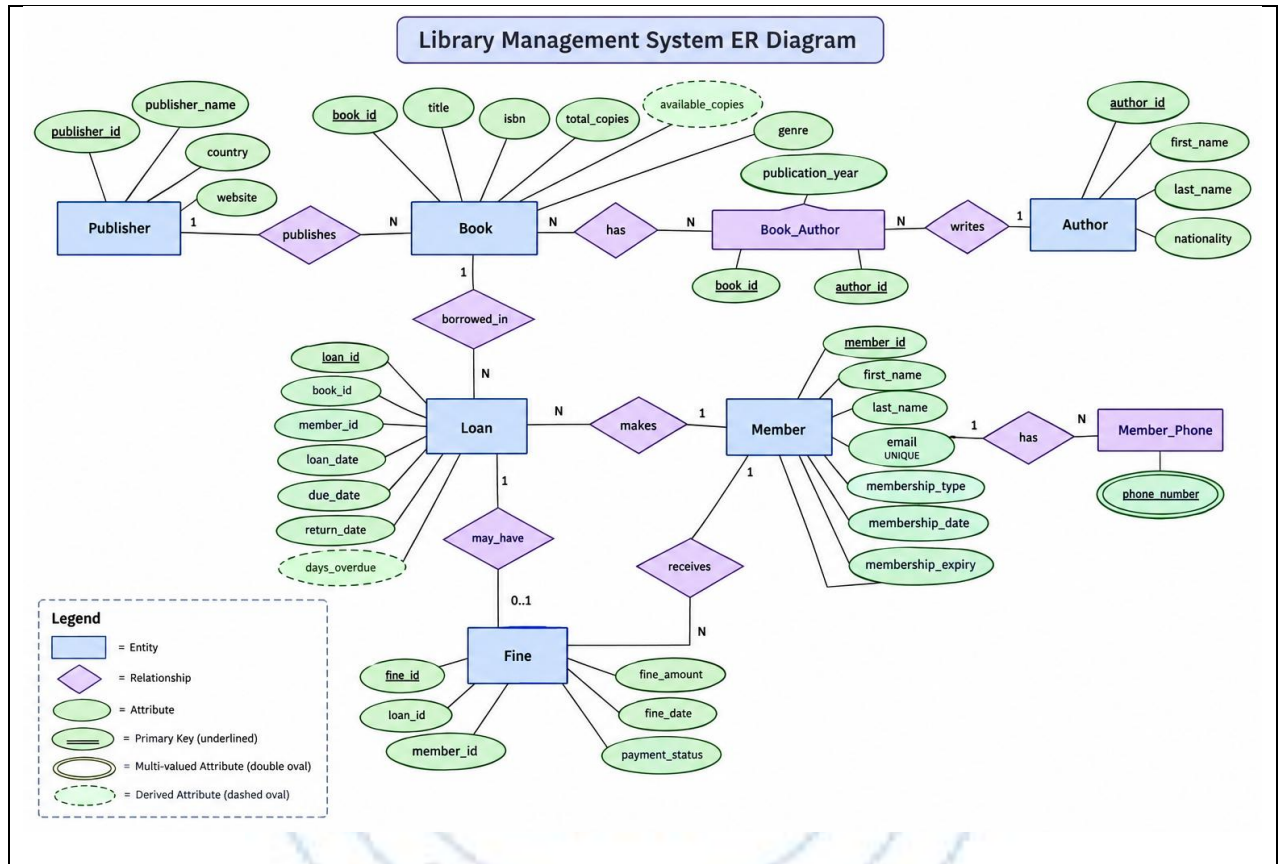


Fig. 3.1: ER Diagram of the Library Management System

3.4 Relationships and Cardinality

Table 3.3: Relationships and Cardinality

Entity 1	Relationship	Entity 2	Cardinality	Participation
Publisher	Publishes	Book	1:N	Partial
Book	Has	Book_Author	N:N	Total
Book_Author	Writes	Author	N:1	Total
Book	Borrowed_In	Loan	1:N	Partial
Loan	Makes	Member	N:1	Total
Member	Has	Member_Phone	1:N	Partial
Loan	May_Have	Fine	1:0..1	Partial
Member	Receives	Fine	1:N	Partial

The ER diagram shows the relationships between the main entities of the Library Management System. A publisher can publish many books, while a book can have multiple authors. Members can borrow books through loans, and a loan may generate a fine. Members can also have multiple phone numbers. The cardinality indicates how many records of one entity can be associated with another entity.

3.5 Relational Database Schema.

Table Name	Column Name	Data Type	Key	Constraints
Publisher	publisher_id	INT	PK	NOT NULL, UNIQUE
Publisher	publisher_name	VARCHAR(100)	—	NOT NULL
Publisher	country	VARCHAR(50)	—	—
Publisher	website	VARCHAR(150)	—	UNIQUE
Book	book_id	INT	PK	NOT NULL, UNIQUE
Book	title	VARCHAR(200)	—	NOT NULL
Book	isbn	VARCHAR(20)	—	UNIQUE, NOT NULL
Book	total_copies	INT	—	NOT NULL, CHECK (total_copies >= 0)
Book	genre	VARCHAR(50)	—	—

Book	publication_year	INT	—	CHECK (publication_year > 0)
Book	publisher_id	INT	FK	REFERENCES Publisher(publisher_id)
Author	author_id	INT	PK	NOT NULL, UNIQUE
Author	first_name	VARCHAR(50)	—	NOT NULL
Author	last_name	VARCHAR(50)	—	NOT NULL
Author	nationality	VARCHAR(50)	—	—
Book_Author	book_id	INT	PK, FK	REFERENCES Book(book_id)
Book_Author	author_id	INT	PK, FK	REFERENCES Author(author_id)
Loan	loan_id	INT	PK	NOT NULL, UNIQUE
Loan	book_id	INT	FK	REFERENCES Book(book_id), NOT NULL
Loan	member_id	INT	FK	REFERENCES Member(member_id), NOT NULL
Loan	loan_date	DATE	—	NOT NULL
Loan	due_date	DATE	—	NOT NULL
Loan	return_date	DATE	—	—
Member	member_id	INT	PK	NOT NULL, UNIQUE
Member	first_name	VARCHAR(50)	—	NOT NULL
Member	last_name	VARCHAR(50)	—	NOT NULL
Member	email	VARCHAR(100)	—	UNIQUE, NOT NULL
Member	membership_type	VARCHAR(30)	—	NOT NULL
Member	membership_date	DATE	—	NOT NULL
Member	membership_expiry	DATE	—	—

Member_Phone	member_id	INT	PK, FK	REFERENCES Member(member_id)
Member_Phone	phone_number	VARCHAR(15)	PK	NOT NULL
Fine	fine_id	INT	PK	NOT NULL, UNIQUE
Fine	loan_id	INT	FK	REFERENCES Loan(loan_id), UNIQUE
Fine	member_id	INT	FK	REFERENCES Member(member_id), NOT NULL
Fine	fine_amount	DECIMAL(10,2)	—	NOT NULL, CHECK (fine_amount >= 0)
Fine	fine_date	DATE	—	NOT NULL
Fine	payment_status	VARCHAR(20)	—	NOT NULL

Table 3.4: Relational Database Schema

Schema Diagram

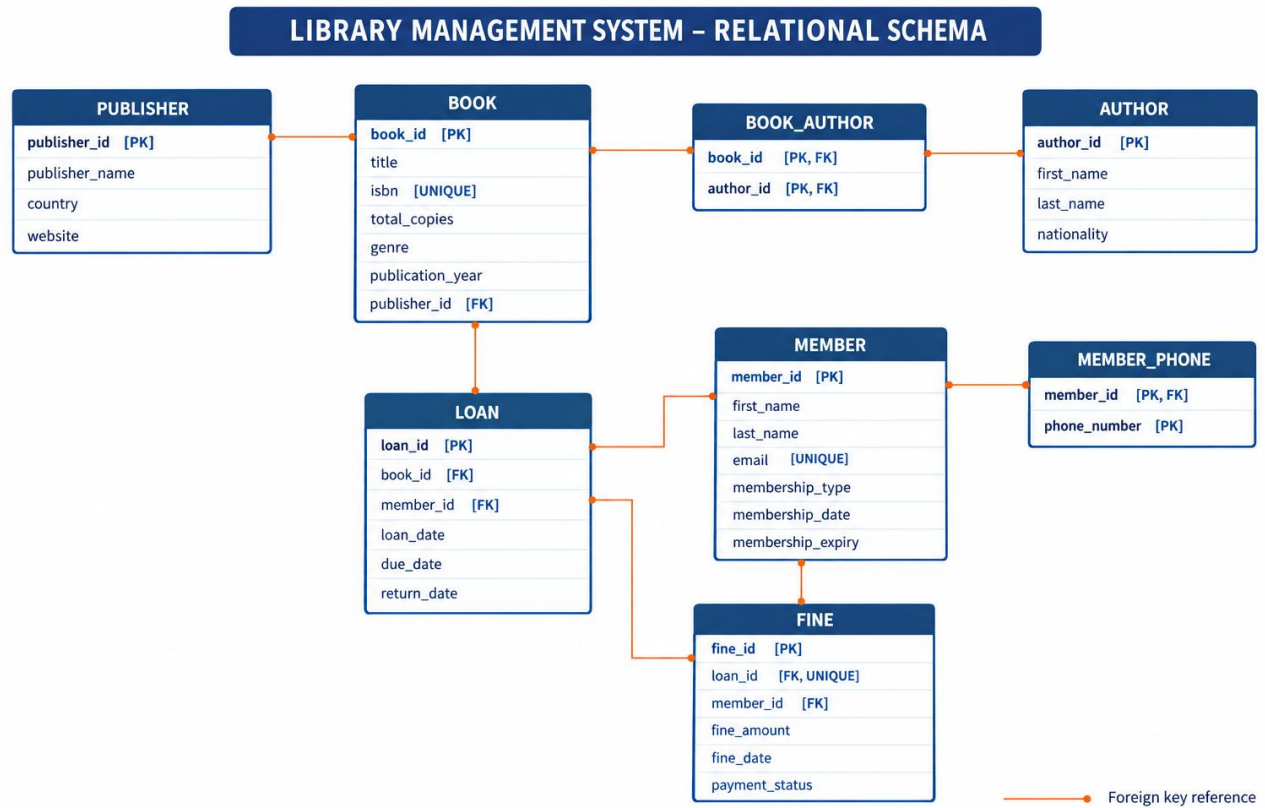


Fig. 3.2: Relational Schema Diagram

3.6 Primary Keys and Foreign Keys

Table 3.5: Primary Key and Foreign Key Definitions

Table	Primary Key	Foreign Key	References (Table.Column)
Publisher	publisher_id	—	—
Book	book_id	publisher_id	Publisher.publisher_id
Author	author_id	—	—
Book_Author	book_id, author_id	book_id, author_id	Book.book_id, Author.author_id
Member	member_id	—	—
Member_Phone	member_id, phone_number	member_id	Member.member_id
Loan	loan_id	book_id, member_id	Book.book_id, Member.member_id
Fine	fine_id	loan_id, member_id	Loan.loan_id, Member.member_id

3.7 Normalization up to 3NF

The initial Library Management System design held information about books, authors, publishers, members, loans and fines in a single unnormalized relation. A possible unnormalized relation is shown below:

LIBRARY_RECORD

(book_id, title, isbn, total_copies, genre, publication_year, publisher_id, publisher_name, country, website, author_id, author_name, nationality, member_id, member_name, email, phone_numbers, membership_type, loan_id, loan_date, due_date, return_date, fine_id, fine_amount, fine_date, payment_status)

First Normal Form (1NF)

The initial relation may contain multiple authors for a book and multiple phone numbers for a member. 1NF requires every attribute to contain a single atomic value and eliminates repeating groups. Therefore, multiple authors are separated into the **Book_Author** table and multiple member phone numbers are separated into **Member_Phone**.

Second Normal Form (2NF)

After 1NF, the **Book_Author** table has the composite key (book_id, author_id). Book-specific information such as title and ISBN depends only on book_id, while author information depends only on author_id. These are partial dependencies on parts of the composite key. Therefore, the design is decomposed into **Book, Author, and Book_Author**, so that non-key attributes depend on the complete key of their respective tables.

Third Normal Form (3NF)

A transitive dependency exists when information about a publisher, author, member, or other entity is stored repeatedly with another entity. For example, publisher_name, country, and website depend on publisher_id, not directly on book_id. Therefore, publisher information is separated into the **Publisher** table. Similarly, author, member and fine information are maintained in their respective tables. This removes unnecessary redundancy and update anomalies.

UNF → 1NF → 2NF → 3NF

Table 3.6: Normalization Steps

Normal Form	Condition Satisfied	Anomaly Removed	Resulting Tables
1NF	All attributes are atomic and there are no repeating groups or multi-valued attributes.	Multiple authors and multiple phone numbers stored in a single record.	Book, Author, Member, Book_Author, Member_Phone, Loan, Fine, Publisher
2NF	No partial dependency exists on part of a composite key. Every non-key attribute depends on the complete primary key.	Book and author details being repeated for every book-author combination.	Book, Author, Book_Author, Member, Member_Phone, Publisher, Loan, Fine
3NF	No transitive dependency exists. Non-key attributes depend only on the primary key.	Publisher, author, member and other details being unnecessarily repeated in related records.	Publisher, Book, Author, Book_Author, Member, Member_Phone, Loan, Fine

After the third step, the schema consists of **eight tables: Publisher, Book, Author, Book_Author, Member, Member_Phone, Loan, and Fine**. The design is normalized up to **3NF**, reducing data redundancy and preventing insertion, update, and deletion anomalies.



CHAPTER 4: DATABASE IMPLEMENTATION

(Deliverables of Review – II)

4.1 Database and Table Creation – DDL

```
CREATE TABLE Publisher (  
    publisher_id INT PRIMARY KEY,  
    publisher_name VARCHAR(100) NOT NULL,  
    country VARCHAR(50),  
    website VARCHAR(150),  
  
    CONSTRAINT uq_publisher_name  
        UNIQUE (publisher_name),  
  
    CONSTRAINT uq_publisher_website  
        UNIQUE (website)  
);  
  
CREATE TABLE Author (  
    author_id INT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    nationality VARCHAR(50)  
);  
  
CREATE TABLE Book (  
    book_id INT PRIMARY KEY,  
    title VARCHAR(200) NOT NULL,  
    isbn VARCHAR(20) NOT NULL,  
    total_copies INT NOT NULL DEFAULT 1,  
    genre VARCHAR(50),  
    publication_year INT,  
    publisher_id INT NOT NULL,  
  
    CONSTRAINT uq_book_isbn  
        UNIQUE (isbn),  
  
    CONSTRAINT chk_total_copies  
        CHECK (total_copies >= 0),  
  
    CONSTRAINT chk_publication_year  
        CHECK (publication_year > 0),  
  
    CONSTRAINT fk_book_publisher  
        FOREIGN KEY (publisher_id)  
        REFERENCES Publisher(publisher_id)  
);
```

```
CREATE TABLE Book_Author (  
    book_id INT NOT NULL,  
    author_id INT NOT NULL,  
  
    PRIMARY KEY (book_id, author_id),  
  
    CONSTRAINT fk_book_author_book  
        FOREIGN KEY (book_id)  
        REFERENCES Book(book_id)  
        ON DELETE CASCADE,  
  
    CONSTRAINT fk_book_author_author  
        FOREIGN KEY (author_id)  
        REFERENCES Author(author_id)  
        ON DELETE CASCADE  
);  
CREATE TABLE Member (  
    member_id INT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) NOT NULL,  
    membership_type VARCHAR(30) NOT NULL,  
    membership_date DATE NOT NULL,  
    membership_expiry DATE,  
  
    CONSTRAINT uq_member_email  
        UNIQUE (email)  
);  
CREATE TABLE Member_Phone (  
    member_id INT NOT NULL,  
    phone_number VARCHAR(15) NOT NULL,  
  
    PRIMARY KEY (member_id, phone_number),  
  
    CONSTRAINT fk_member_phone_member  
        FOREIGN KEY (member_id)  
        REFERENCES Member(member_id)  
        ON DELETE CASCADE  
);  
CREATE TABLE Loan (  
    loan_id INT PRIMARY KEY,  
    book_id INT NOT NULL,  
    member_id INT NOT NULL,  
    loan_date DATE NOT NULL,  
    due_date DATE NOT NULL,  
    return_date DATE,
```

```

CONSTRAINT fk_loan_book
    FOREIGN KEY (book_id)
    REFERENCES Book(book_id),

CONSTRAINT fk_loan_member
    FOREIGN KEY (member_id)
    REFERENCES Member(member_id),

CONSTRAINT chk_due_date
    CHECK (due_date >= loan_date),

CONSTRAINT chk_return_date
    CHECK (
        return_date IS NULL
        OR return_date >= loan_date
    )
);

CREATE TABLE Fine (
    fine_id INT PRIMARY KEY,
    loan_id INT NOT NULL,
    member_id INT NOT NULL,
    fine_amount DECIMAL(10,2) NOT NULL DEFAULT 0,
    fine_date DATE NOT NULL,
    payment_status VARCHAR(20) NOT NULL DEFAULT 'Unpaid',

    CONSTRAINT uq_fine_loan
        UNIQUE (loan_id),

    CONSTRAINT fk_fine_loan
        FOREIGN KEY (loan_id)
        REFERENCES Loan(loan_id),

    CONSTRAINT fk_fine_member
        FOREIGN KEY (member_id)
        REFERENCES Member(member_id),

    CONSTRAINT chk_fine_amount
        CHECK (fine_amount >= 0),

    CONSTRAINT chk_payment_status
        CHECK (payment_status IN ('Paid', 'Unpaid'))
);

```

The script executed without errors and **created eight tables**. The output of the table listing is shown below.

Tables in library management system
Author
Book
Book_Author
Fine
Loan
Member
Member_Phone
Publisher

Fig. 4.1: Successful Creation of Tables



4.2 Sample Data Insertion – DML

Sample data was inserted into all **eight tables** of the Library Management System. The data is selected to exercise the relationships and constraints, including books with multiple authors, members with multiple phone numbers, loans, and fines.

INSERT INTO Publisher

(publisher_id, publisher_name, country, website)

VALUES

(1, 'Penguin Books', 'India', 'https://www.penguin.co.in'),

(2, 'Oxford University Press', 'UK', 'https://global.oup.com'),

(3, 'McGraw Hill', 'USA', 'https://www.mheducation.com'),

(4, 'Pearson Education', 'UK', 'https://www.pearson.com');

INSERT INTO Author

(author_id, first_name, last_name, nationality)

VALUES

(1, 'Ravi', 'Kumar', 'Indian'),

(2, 'Anita', 'Sharma', 'Indian'),

(3, 'John', 'Smith', 'American'),

(4, 'Robert', 'Martin', 'American'),

(5, 'Priya', 'Reddy', 'Indian');

INSERT INTO Member

(member_id, first_name, last_name, email, membership_type,

membership_date, membership_expiry)

VALUES

(101, 'Rahul', 'Reddy', 'rahul.reddy@gmail.com',

'Regular', '2026-01-10', '2027-01-10'),

(102, 'Priya', 'Sharma', 'priya.sharma@gmail.com',

```
'Premium', '2026-02-15', '2027-02-15'),  
(103, 'Arjun', 'Kumar', 'arjun.kumar@gmail.com', 'Regular', '2026-03-01', '2027-03-01'),  
(104, 'Sneha', 'Patel', 'sneha.patel@gmail.com'  
'Student', '2026-04-05', '2027-04-05'), (105, 'Karthik', 'Varma', 'karthik.varma@gmail.com',  
'Premium', '2026-05-12', '2027-05-12');
```

```
INSERT INTO Book
```

```
(book_id, title, isbn, total_copies, genre, publication_year, publisher_id)
```

```
VALUES
```

```
(1001, 'Database Management Systems', 'ISBN1001', 5, 'Database', 2024, 1),  
(1002, 'Data Structures and Algorithms', 'ISBN1002', 4, 'Computer Science', 2023, 2),  
(1003, 'Computer Networks', 'ISBN1003', 3, 'Networking', 2022, 3),  
(1004, 'Operating Systems', 'ISBN1004', 6, 'Computer Science', 2024, 1),  
(1005, 'Machine Learning', 'ISBN1005', 4, 'Artificial Intelligence', 2025, 4),  
(1006, 'Web Technologies', 'ISBN1006', 5, 'Web Development', 2023, 3),  
(1007, 'Clean Code', 'ISBN1007', 3, 'Programming', 2021, 2);
```

```
INSERT INTO Book_Author
```

```
(book_id, author_id)
```

```
VALUES
```

```
(1001, 1),  
(1001, 2),  
(1002, 3),  
(1003, 4),  
(1004, 1),  
(1005, 5),  
(1006, 2),  
(1007, 4);
```

```
INSERT INTO Member_Phone (member_id, phone_number)
```

```
VALUES
```

```
(101, '9876543210'),
```

```
(101, '9440011223'),
```

```
(102, '9848012345'),
```

```
(103, '9701122334'),
```

```
(104, '9885566778'),
```

```
(105, '9963321456');
```

```
INSERT INTO Loan
```

```
(loan_id, book_id, member_id, loan_date, due_date, return_date)
```

```
VALUES
```

```
(5001, 1001, 101, '2026-08-01', '2026-08-15', '2026-08-14'),
```

```
(5002, 1002, 102, '2026-08-05', '2026-08-19', NULL),
```

```
(5003, 1003, 103, '2026-08-10', '2026-08-24', '2026-08-23'),
```

```
(5004, 1004, 104, '2026-08-15', '2026-08-29', NULL),
```

```
(5005, 1005, 105, '2026-09-01', '2026-09-15', '2026-09-14'),
```

```
(5006, 1006, 101, '2026-09-05', '2026-09-19', NULL);
```

```
INSERT INTO Fine
```

```
(fine_id, loan_id, member_id, fine_amount, fine_date, payment_status)
```

```
VALUES
```

```
(9001, 5002, 102, 50.00, '2026-08-20', 'Unpaid'),
```

```
(9002, 5004, 104, 75.00, '2026-08-30', 'Paid'),
```

```
(9003, 5006, 101, 30.00, '2026-09-20', 'Unpaid');
```

--DML EXAMPLES

UPDATE Fine

SET payment status = 'Paid'

WHERE fine_id = 9001;

UPDATE Book

SET total_copies = 7

WHERE book_id = 1004;

DELETE FROM Member_Phone

WHERE member_id = 101

AND phone_number = '9440011223';

Table 4.1: Sample Data Summary

S.No	Table Name	No. of Records Inserted	Remarks
1	Publisher	4	Publisher details inserted
2	Author	5	Author details inserted
3	Book	7	Book details with publisher references inserted
4	Book_Author	8	Book-author relationships inserted
5	Member	5	Library member details inserted
6	Member_Phone	6	Phone numbers inserted; one member has two numbers
7	Loan	6	Book borrowing records inserted
8	Fine	3	Fine records with payment status inserted

Fig. 4.2: Sample Data Inserted into the Tables

4.3 Constraints and Data Integrity

Table 4.2: Constraints and Data Integrity Rules

Constraint / Validation	Applied On (Table.Column)	Purpose
Primary Key	Publisher.publisher_id	Uniquely identifies each publisher
Primary Key	Book.book_id	Uniquely identifies each book
Primary Key	Author.author_id	Uniquely identifies each author
Composite Primary Key	Book_Author.book_id, author_id	Uniquely identifies each book-author relationship
Primary Key	Member.member_id	Uniquely identifies each member
Composite Primary	Member_Phone.member_id,	Allows multiple unique phone numbers

Key	phone_number	for a member
Primary Key	Loan.loan_id	Uniquely identifies each loan
Primary Key	Fine.fine_id	Uniquely identifies each fine
NOT NULL	Book.title, isbn, total_copies	Prevents missing essential book information
NOT NULL	Member.first_name, last_name, email	Ensures required member information is provided
UNIQUE	Book.isbn	Prevents duplicate ISBN values
UNIQUE	Member.email	Prevents duplicate member email addresses
UNIQUE	Fine.loan_id	Allows at most one fine for each loan
FOREIGN KEY	Book.publisher_id	Ensures every book references an existing publisher
FOREIGN KEY	Loan.book_id	Ensures every loan references an existing book
FOREIGN KEY	Loan.member_id	Ensures every loan references an existing member
FOREIGN KEY	Fine.loan_id, Fine.member_id	Maintains valid loan and member references
CHECK	Book.total_copies >= 0	Prevents negative book quantities
CHECK	Fine.fine_amount >= 0	Prevents negative fine amounts
CHECK	Fine.payment_status	Allows only Paid or Unpaid
CHECK	Loan.due_date >= loan_date	Ensures valid loan dates
CHECK	Loan.return_date >= loan_date	Prevents an invalid return date

4.4 Data Validation Rules

To confirm that the constraints actually work, three invalid statements were executed deliberately. In each case, the database rejected the statement and no row was inserted.

Referential Integrity — loan for a book that does not exist:

```
INSERT INTO Loan
```

```
VALUES (99, 999, 1, '2026-09-20', '2026-09-27', NULL);
```



```
INSERT INTO Loan
VALUES (99, 999, 1, '2026-09-20', '2026-09-27', NULL);
```

error: insert or update on table "loan" violate

(b) CHECK Constraint — negative number of book copies:

INSERT INTO Book

VALUES (99, 'Test Book', 'ISBN999', -5, 'Testing', 2026, 1);



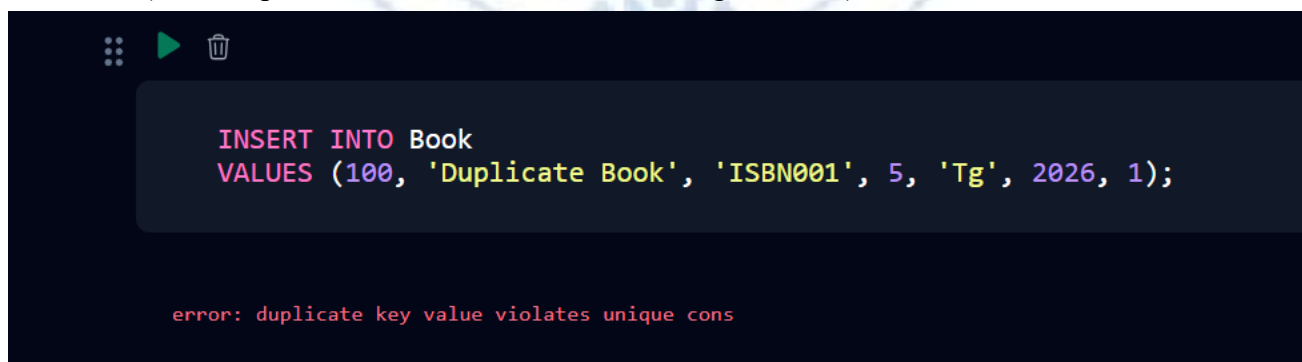
```
INSERT INTO Book
VALUES (99, 'Test Book', 'ISBN999', -5, 'Testing', 2026, 1);
```

error: new row for relation "book" violates che

(c) UNIQUE Constraint — duplicate book ISBN:

INSERT INTO Book

VALUES (100, 'Duplicate Book', 'ISBN001', 5, 'Testing', 2026, 1);



```
INSERT INTO Book
VALUES (100, 'Duplicate Book', 'ISBN001', 5, 'Tg', 2026, 1);
```

error: duplicate key value violates unique cons

Fig. 4.3: Referential Integrity Enforced by the Database

CHAPTER 5: SQL OPERATIONS AND QUERIES

Seventeen queries were executed against the sample data. For each query the purpose, the SQL statement and the actual output are given.

5.1 DQL – Basic Retrieval Queries

Q1. Display all books with their ISBN, genre, and publisher ID.

SQL:

```
SELECT book_id, title, isbn, genre, publisher_id
```

```
FROM Book
```

```
ORDER BY book_id;
```

OUTPUT:

book_id	title	isbn	genre	publisher_id
100	Duplicate Book	ISBN001	Testing	1
1001	Database Management Systems	ISBN1001	Database	1
1002	Data Structures and Algorithms	ISBN1002	Computer Science	2
1003	Computer Networks	ISBN1003	Networking	3
1004	Operating Systems	ISBN1004	Computer Science	1
1005	Machine Learning	ISBN1005	Artificial Intelligence	4
1006	Web Technologies	ISBN1006	Web Development	3
1007	Clean Code	ISBN1007	Programming	2

Q2. Display books that have more than 5 total copies.

SQL:

```
SELECT book_id, title, total_copies
```

```
FROM Book
```

```
WHERE total_copies > 5
```

```
ORDER BY title;
```

OUTPUT:

book_id	title	total_copies
1002	Data Structures and Algorithms	4
1001	Database Management Systems	5
100	Duplicate Book	5
1005	Machine Learning	4
1004	Operating Systems	6
1006	Web Technologies	5

Q3. Display loans that have not yet been returned.

SQL:

```
SELECT loan_id, book_id, member_id, loan_date, due_date
FROM Loan
WHERE return_date IS NULL
ORDER BY loan_id;
```

OUTPUT:

loan_id	book_id	member_id	loan_date	due_date
5002	1002	102	"2026-08-05T00:00:00.000Z"	"2026-08-19T00:00:00.000Z"
5004	1004	104	"2026-08-15T00:00:00.000Z"	"2026-08-29T00:00:00.000Z"
5006	1006	101	"2026-09-05T00:00:00.000Z"	"2026-09-19T00:00:00.000Z"

5.2 Join Operations

Q4. Natural Join -Display books with their publisher details using common columns.

SQL:

```
SELECT book_id, title, publisher_name
```

```
FROM Book
```

```
NATURAL JOIN Publisher;
```

OUTPUT:

book_id	title	publisher_name
1001	Database Management Systems	Penguin Books
1002	Data Structures and Algorithms	Oxford University Press
1003	Computer Networks	McGraw Hill
1004	Operating Systems	Penguin Books
1005	Machine Learning	Pearson Education
1006	Web Technologies	McGraw Hill
1007	Clean Code	Oxford University Press
100	Duplicate Book	Penguin Books

Q5. Equi Join - Display books and their corresponding publishers.

SQL:

```
SELECT b.book_id, b.title, p.publisher_name
```

```
FROM Book b, Publisher p
```

```
WHERE b.publisher_id = p.publisher_id;
```

OUTPUT:

book_id	title	publisher_name
1001	Database Management Systems	Penguin Books
1002	Data Structures and Algorithms	Oxford University Press
1003	Computer Networks	McGraw Hill
1004	Operating Systems	Penguin Books
1005	Machine Learning	Pearson Education
1006	Web Technologies	McGraw Hill
1007	Clean Code	Oxford University Press
100	Duplicate Book	Penguin Books

Q6. Inner Join - Display books along with publisher names

SQL:

```
SELECT b.book_id, b.title, p.publisher_name
```

```
FROM Book b
```

```
INNER JOIN Publisher p
```

```
ON b.publisher_id = p.publisher_id;
```

OUTPUT:

book_id	title	publisher_name
1001	Database Management Systems	Penguin Books
1002	Data Structures and Algorithms	Oxford University Press
1003	Computer Networks	McGraw Hill
1004	Operating Systems	Penguin Books
1005	Machine Learning	Pearson Education
1006	Web Technologies	McGraw Hill
1007	Clean Code	Oxford University Press
100	Duplicate Book	Penguin Books

Q7. Left Outer Join - Display all books, including books without a matching publisher.

SQL:

```
SELECT b.book_id, b.title, p.publisher_name
```

```
FROM Book b
```

```
LEFT JOIN Publisher p
```

```
ON b.publisher_id = p.publisher_id;
```

OUTPUT:

book_id	title	publisher_name
1001	Database Management Systems	Penguin Books
1002	Data Structures and Algorithms	Oxford University Press
1003	Computer Networks	McGraw Hill
1004	Operating Systems	Penguin Books
1005	Machine Learning	Pearson Education
1006	Web Technologies	McGraw Hill
1007	Clean Code	Oxford University Press
100	Duplicate Book	Penguin Books

Q8. Right Outer Join - Display all publishers, including publishers without any books.

SQL:

```
SELECT b.book_id, b.title, p.publisher_name
```

```
FROM Book b
```

```
RIGHT JOIN Publisher p
```

```
ON b.publisher_id = p.publisher_id;
```

OUTPUT:

book_id	title	publisher_name
1001	Database Management Systems	Penguin Books
1002	Data Structures and Algorithms	Oxford University Press
1003	Computer Networks	McGraw Hill
1004	Operating Systems	Penguin Books
1005	Machine Learning	Pearson Education
1006	Web Technologies	McGraw Hill
1007	Clean Code	Oxford University Press
100	Duplicate Book	Penguin Books

Q9. Full Outer Join - Display all books and publishers, including unmatched records from both tables.

SQL:

```
SELECT b.book_id, b.title, p.publisher_name
FROM Book b
FULL OUTER JOIN Publisher p
ON b.publisher_id = p.publisher_id;
```

OUTPUT:

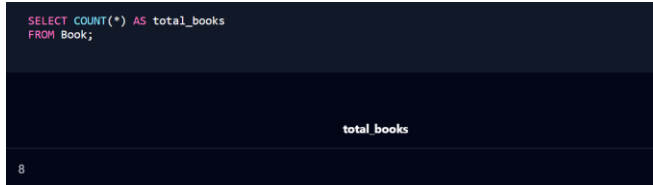
book_id	title	publisher_name
1001	Database Management Systems	Penguin Books
1002	Data Structures and Algorithms	Oxford University Press
1003	Computer Networks	McGraw Hill
1004	Operating Systems	Penguin Books
1005	Machine Learning	Pearson Education
1006	Web Technologies	McGraw Hill
1007	Clean Code	Oxford University Press
100	Duplicate Book	Penguin Books

5.3 Aggregate Functions, GROUP BY and HAVING

Q10. Count the total number of books

```
SELECT COUNT(*) AS total_books  
FROM Book;
```

OUTPUT:



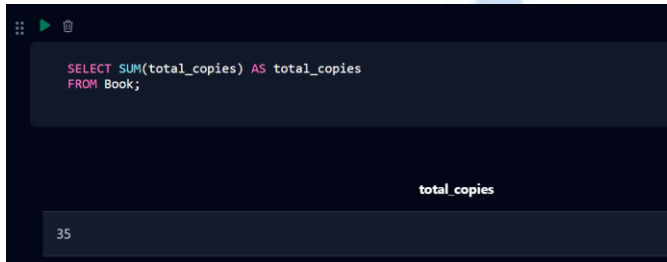
The screenshot shows a SQL query editor with the query: `SELECT COUNT(*) AS total_books FROM Book;`. Below the query, the result is displayed in a table with one column named `total_books` and one row containing the value `8`.

total_books
8

Q11. Find the total number of book copies

```
SELECT SUM(total_copies) AS total_copies  
FROM Book;
```

OUTPUT:



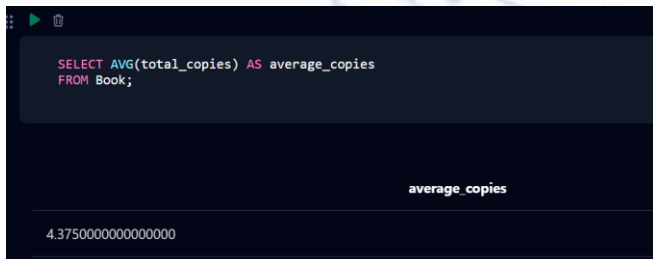
The screenshot shows a SQL query editor with the query: `SELECT SUM(total_copies) AS total_copies FROM Book;`. Below the query, the result is displayed in a table with one column named `total_copies` and one row containing the value `35`.

total_copies
35

Q12. Find the average number of copies per book

```
SELECT AVG(total_copies) AS average_copies  
FROM Book;
```

OUTPUT:



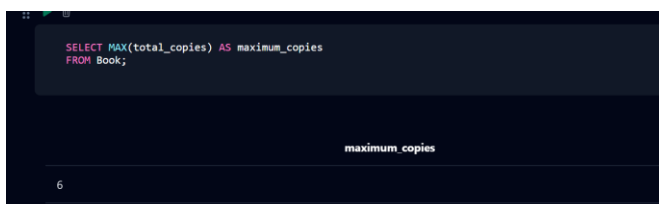
The screenshot shows a SQL query editor with the query: `SELECT AVG(total_copies) AS average_copies FROM Book;`. Below the query, the result is displayed in a table with one column named `average_copies` and one row containing the value `4.3750000000000000`.

average_copies
4.3750000000000000

Q13. Find the maximum number of copies of any book

```
SELECT MAX(total_copies) AS maximum_copies  
FROM Book;
```

OUTPUT:



The screenshot shows a SQL query editor with the query: `SELECT MAX(total_copies) AS maximum_copies FROM Book;`. Below the query, the result is displayed in a table with one column named `maximum_copies` and one row containing the value `6`.

maximum_copies
6

Q14. Find the minimum number of copies of any book

```
SELECT MIN(total_copies) AS minimum_copies  
FROM Book;
```

OUTPUT:



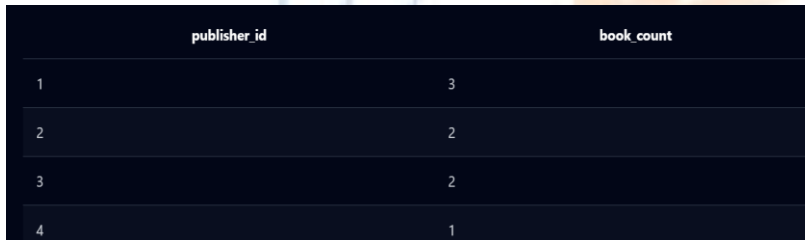
The screenshot shows a SQL query execution interface. The query is: `SELECT MIN(total_copies) AS minimum_copies FROM Book;`. The result is displayed in a table with one column labeled `minimum_copies` and one row with the value `3`.

minimum_copies
3

Q15. Count books published by each publisher

```
SELECT publisher_id, COUNT(*) AS book_count  
FROM Book  
GROUP BY publisher_id  
ORDER BY publisher_id;
```

OUTPUT:



The screenshot shows a SQL query execution interface. The query is: `SELECT publisher_id, COUNT(*) AS book_count FROM Book GROUP BY publisher_id ORDER BY publisher_id;`. The result is displayed in a table with two columns: `publisher_id` and `book_count`. The data is as follows:

publisher_id	book_count
1	3
2	2
3	2
4	1

Q16. Calculate total copies supplied by each publisher

```
SELECT publisher_id, SUM(total_copies) AS total_copies  
FROM Book  
GROUP BY publisher_id  
ORDER BY publisher_id;
```

OUTPUT:



The screenshot shows a SQL query execution interface. The query is: `SELECT publisher_id, SUM(total_copies) AS total_copies FROM Book GROUP BY publisher_id ORDER BY publisher_id;`. The result is displayed in a table with two columns: `publisher_id` and `total_copies`. The data is as follows:

publisher_id	total_copies
1	16
2	7
3	8
4	4

Q17. Display publishers having more than one book

```
SELECT publisher_id, COUNT(*) AS book_count
FROM Book
GROUP BY publisher_id
HAVING COUNT(*) > 1 ORDER BY publisher_id;
```

OUTPUT:

publisher_id	book_count
1	3
2	2
3	2

5.4 Nested and Correlated Subqueries

Q18. Nested Subquery - Displays books whose number of copies is greater than the average number of copies of all books.

SQL:

```
SELECT book_id, title, total_copies
FROM Book
WHERE total_copies > (
    SELECT AVG(total_copies)
    FROM Book
)
ORDER BY book_id;
```

OUTPUT:

book_id	title	total_copies
100	Duplicate Book	5
1001	Database Management Systems	5
1004	Operating Systems	6
1006	Web Technologies	5

Q19. Correlated Subquery - Displays members who have at least one loan record in the Loan table.

SQL:

```
SELECT m.member_id, m.first_name, m.last_name
FROM Member m
WHERE EXISTS (
```

```

SELECT 1
FROM Loan l
WHERE l.member_id = m.member_id
)
ORDER BY m.member_id;

```

OUTPUT:

member_id	first_name	last_name
101	Rahul	Reddy
102	Priya	Sharma
103	Arjun	Kumar
104	Sneha	Patel
105	Karthik	Varma

5.5 Views and Relational Set Operations

Two views were created for the Library Management System. The v_book_publisher view combines information from the Book and Publisher tables and is used for reporting. The v_book_summary view uses aggregation to summarize the number of books by genre and is therefore non-updatable.

View 1: Book-Publisher View

```

CREATE VIEW v_book_publisher AS
SELECT b.book_id, b.title, b.isbn, p.publisher_name, b.genre
FROM Book b
JOIN Publisher p
ON b.publisher_id = p.publisher_id;

```

View 2: Book Summary View

```

CREATE VIEW v_book_summary AS
SELECT genre,
COUNT(book_id) AS total_books
FROM Book

```

GROUP BY genre;

Q20. Query the v_book_publisher view

SQL:

```
SELECT book_id, title, publisher_name, genre
```

```
FROM v_book_publisher
```

```
ORDER BY book_id;
```

Output:

book_id	title	publisher_name	genre
100	Duplicate Book	Penguin Books	Testing
1001	Database Management Systems	Penguin Books	Database
1002	Data Structures and Algorithms	Oxford University Press	Computer Science
1003	Computer Networks	McGraw Hill	Networking
1004	Operating Systems	Penguin Books	Computer Science
1005	Machine Learning	Pearson Education	Artificial Intelligence
1006	Web Technologies	McGraw Hill	Web Development
1007	Clean Code	Oxford University Press	Programming

Q21. Query the v_book_summary view

SQL:

```
SELECT genre, total_books
```

```
FROM v_book_summary
```

```
ORDER BY total_books DESC;
```

Output:

genre	total_books
Computer Science	2
Database	1
Testing	1
Networking	1
Artificial Intelligence	1
Web Development	1
Programming	1

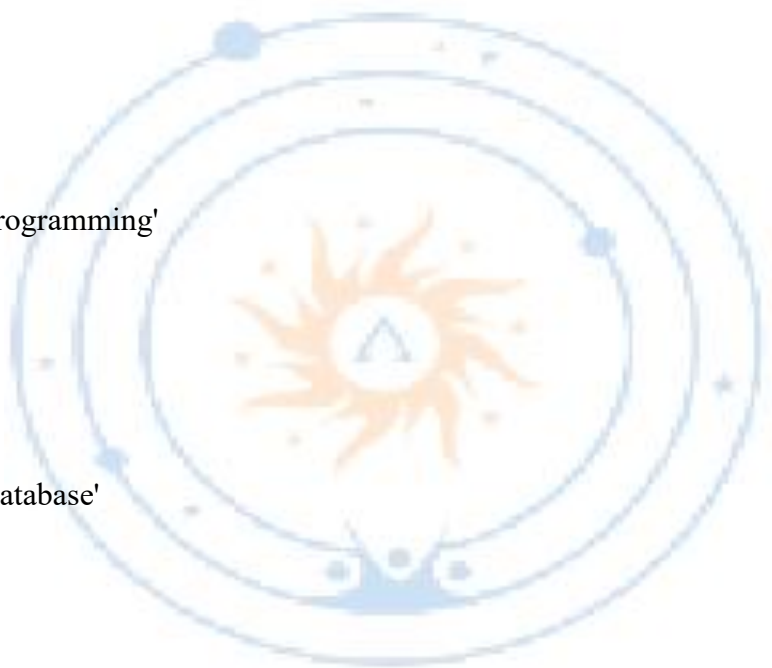
Q22. UNION – Genres with books from selected categories

SQL:

```

SELECT genre
FROM Book
WHERE genre = 'Programming'
UNION
SELECT genre
FROM Book
WHERE genre = 'Database'
ORDER BY genre;

```



Output:

genre
Database
Programming

Q23. INTERSECT – Common genres between two book groups

SQL:

```

SELECT genre
FROM Book
WHERE total_copies >= 5

INTERSECT

SELECT genre
FROM Book
WHERE publication_year >= 2020

ORDER BY genre;

```

OUTPUT:

genre
Computer Science
Database
Testing
Web Development

5.6 DCL and TCL Operations

Data Control Language commands were used to provide and remove access permissions for a library user, while Transaction Control Language commands were used to demonstrate transaction management using COMMIT, SAVEPOINT, and ROLLBACK.

(a) DCL — GRANT and REVOKE:

```
CREATE USER exam_cell WITH PASSWORD 'exam@123';
```

```
GRANT SELECT ON Book TO exam_cell;
```

```
GRANT SELECT ON Loan TO exam_cell;
```

```
GRANT INSERT, UPDATE ON Fine TO exam_cell;
```

```
REVOKE INSERT, UPDATE ON Fine FROM exam_cell;
```

(b) TCL — COMMIT, SAVEPOINT and ROLLBACK:

```
BEGIN;
```

```
UPDATE Fine
```

```
SET fine_amount = 50
```

```
WHERE fine_id = 1;
```

```
SAVEPOINT after_first_update;
```

```
UPDATE Fine
```

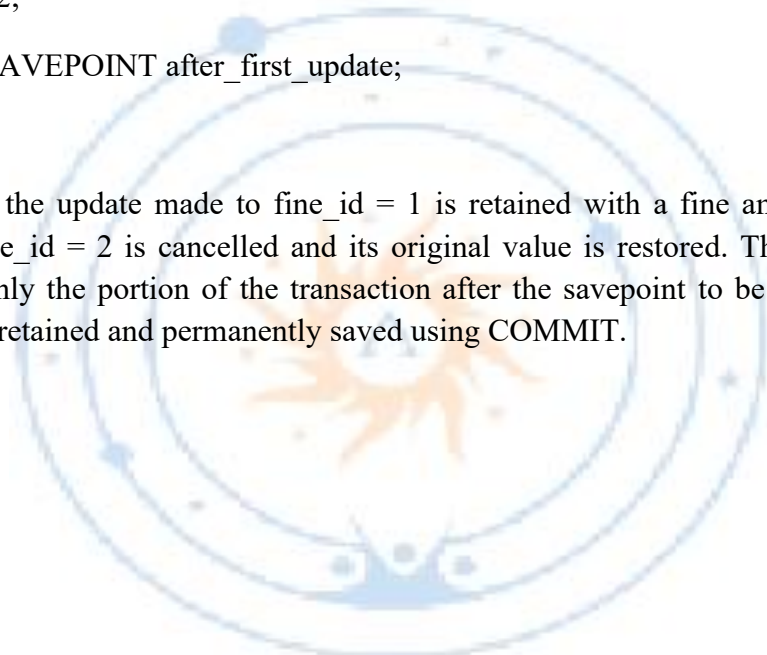
```
SET fine_amount = 100
```

```
WHERE fine_id = 2;
```

```
ROLLBACK TO SAVEPOINT after_first_update;
```

```
COMMIT;
```

After the rollback, the update made to fine_id = 1 is retained with a fine amount of 50, while the update made to fine_id = 2 is cancelled and its original value is restored. This demonstrates that a savepoint allows only the portion of the transaction after the savepoint to be rolled back, while the earlier changes are retained and permanently saved using COMMIT.



CHAPTER 6: MODULES AND FUNCTIONALITIES

The Library Management System is divided into major modules to manage books, authors, publishers, members, loans, and fines efficiently. Each module performs specific operations on the corresponding database tables and helps maintain accurate and organized library records

Table 6.1: Modules and Their Functionalities

S.No	Module Name	Functionality	Tables Involved
1	Book Management	Adds, updates, searches, and displays book details and available copies.	Book
2	Author Management	Stores and manages author information and their details.	Author
3	Publisher Management	Maintains publisher information such as name, country, and website.	Publisher
4	Book-Author Management	Links books with their respective authors, including books with multiple authors.	Book, Author, Book_Author
5	Member Management	Registers members and manages their personal and membership details.	Member
6	Member Phone Management	Stores and manages multiple phone numbers associated with members.	Member, Member_Phone
7	Loan Management	Records book issue and return transactions and maintains due dates.	Loan, Book, Member
8	Fine Management	Calculates and records fines for overdue books and tracks payment status.	Fine, Loan, Member
9	Book Search Module	Allows users to search books using title, ISBN, genre, or publisher.	Book, Publisher
10	Availability Management	Checks the number of copies available for borrowing.	Book, Loan
11	Library Reports	Generates summary information such as total books, books by genre, and loan details.	Book, Loan, Publisher, Fine
12	Data Validation	Enforces primary keys, foreign keys, unique values, NOT NULL, and CHECK constraints.	All Tables

The Library Management System consists of modules for managing books, authors, publishers, members, loans, and fines. It also provides book search and availability checking features. The system maintains relationships between books and authors, stores member phone numbers, generates library reports, and uses database constraints to ensure accurate and consistent data.

CHAPTER 7: RESULTS – SAMPLE OUTPUTS AND SCREENSHOTS

The principal reports produced by the Library Management System are shown below. These reports are generated using SQL queries and database views to provide useful information about books, publishers, loans, and fines.

Report 1 — Book-wise Publisher Report

```
SELECT b.book_id, b.title, p.publisher_name, b.genre
FROM Book b INNER JOIN Publisher p
ON b.publisher_id = p.publisher_id
ORDER BY b.book_id;
```

OUTPUT:

book_id	title	publisher_name	genre
100	Duplicate Book	Penguin Books	Testing
1001	Database Management Systems	Penguin Books	Database
1002	Data Structures and Algorithms	Oxford University Press	Computer Science
1003	Computer Networks	McGraw Hill	Networking
1004	Operating Systems	Penguin Books	Computer Science
1005	Machine Learning	Pearson Education	Artificial Intelligence
1006	Web Technologies	McGraw Hill	Web Development
1007	Clean Code	Oxford University Press	Programming

Fig. 7.1: Book-wise Publisher Report Output

Report 2 — Genre-wise Book Summary

```
SELECT genre, total_books
FROM v_book_summary
ORDER BY total_books DESC;
```

OUTPUT:

genre	total_books
Computer Science	2
Database	1
Testing	1
Networking	1
Artificial Intelligence	1
Web Development	1
Programming	1

Fig. 7.2: Genre-wise Book Summary

The first report combines book and publisher information using an INNER JOIN. The second report combines member and loan information to display borrowing details, including due dates and returned or currently issued books. These outputs demonstrate that the database can efficiently organize and retrieve library information



CHAPTER 8: TESTING AND DATA VALIDATION

The Library Management System was tested using different test cases to verify data validity, database constraints, referential integrity, and correctness of SQL queries. Both valid and invalid inputs were tested to ensure that the database accepts correct data and rejects incorrect data.

Table 8.1: Test Cases and Results

TC No.	Test Case Description	Input / Action	Expected Result	Actual Result	Status
TC01	Insert valid publisher	Insert a new publisher record	Record inserted successfully	Record inserted	Pass
TC02	Insert valid book	Insert a book with valid publisher ID	Book inserted successfully	Book inserted	Pass
TC03	Duplicate ISBN	Insert a book with existing ISBN	Duplicate ISBN rejected	Unique constraint error	Pass
TC04	Negative book copies	Insert total_copies = -5	Record rejected	CHECK constraint error	Pass
TC05	Invalid publisher reference	Insert book with non-existing publisher_id	Record rejected	Foreign key error	Pass
TC06	Insert valid author	Insert valid author details	Author inserted successfully	Author inserted	Pass
TC07	Duplicate member email	Insert member with existing email	Record rejected	Unique constraint error	Pass
TC08	Null book title	Insert book without title	Record rejected	NOT NULL error	Pass
TC09	Valid book-author mapping	Insert valid book and author IDs	Mapping inserted	Mapping inserted	Pass
TC10	Invalid book-author mapping	Insert non-existing book ID	Record rejected	Foreign key error	Pass

The test results show that the Library Management System correctly handles valid and invalid data. Primary key, foreign key, UNIQUE, NOT NULL, and CHECK constraints prevent incorrect entries. SQL queries, joins, aggregate operations, and transaction commands were also tested successfully. Therefore, the database maintains data integrity and produces the expected results.

CHAPTER 9: GITHUB REPOSITORY AND TEAM CONTRIBUTION

9.1 Repository Details

GitHub Repository Link: _____

The final SQL scripts and all project files were organized and pushed to the team GitHub repository. The repository contains the database creation scripts, sample data, SQL queries, ER diagram, project documentation, screenshots, and README file. This provides a centralized location for maintaining, sharing, and reviewing the complete Library Management System project.

9.2 Team Responsibilities and Contribution

Table 9.1: Team Responsibilities

S.No	Student Name	Roll Number	Role / Responsibility	Modules Contributed
1	Y.RENUKA LAHARI	25B11AIC88	Database Design & SQL Development	Book, Author, Publisher, Book-Author
2	CH.NAKASHIMA	25B11AI231	Member & Loan Management	Member, Member Phone, Loan
3	K.SARANYA	25B11AI458	Fine Management & Testing	Fine, Data Validation, Testing
4	CH. HARISWAR	25B11AI227	Documentation & GitHub Management	Reports, Screenshots, ER Diagram, GitHub

9.3 Review-wise Progress

Table 9.2: Review-wise Progress Record

Review	Date	Deliverables Presented	Remarks of the Guide

CHAPTER 10: CONCLUSION AND FUTURE ENHANCEMENTS

10.1 Conclusion

The Library Management System was successfully designed and implemented to provide a systematic and efficient method for managing library records. The database contains **8 tables: Publisher, Author, Book, Book_Author, Member, Member_Phone, Loan, and Fine**, which together represent the major activities of a library. Relationships between these tables were properly established using primary keys and foreign keys.

The database was designed using normalization principles to minimize data redundancy and maintain consistency. Appropriate **NOT NULL, UNIQUE, CHECK, PRIMARY KEY, and FOREIGN KEY** constraints were applied to ensure data accuracy and integrity. The system supports the management of book details, author and publisher information, member records, book issue and return details, and fines.

Different SQL operations such as **DDL, DML, DQL, DCL, and TCL** were implemented and tested. Advanced queries using **JOINS, aggregate functions, GROUP BY, HAVING, nested and correlated subqueries, views, and set operations** were also performed. Transaction operations such as COMMIT, SAVEPOINT, and ROLLBACK were demonstrated to ensure controlled data updates.

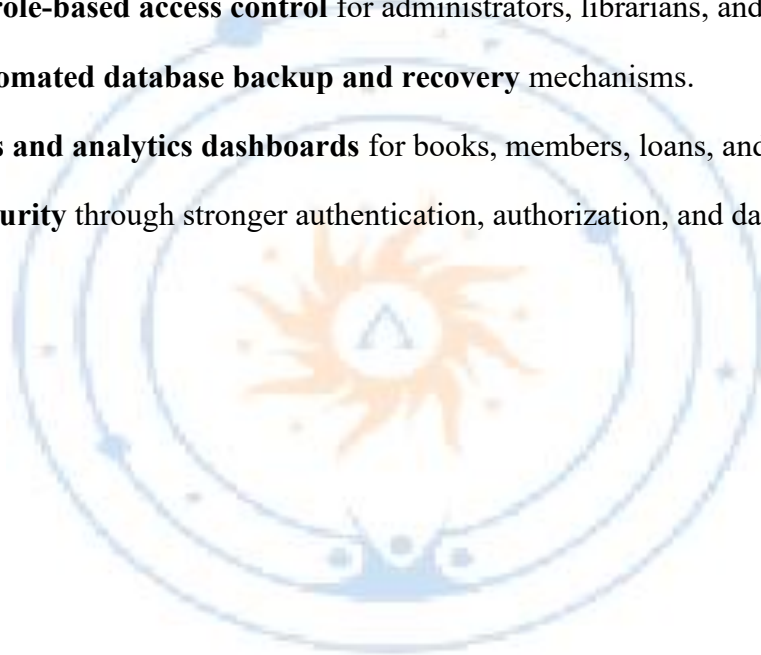
The testing phase confirmed that the database correctly handles valid data and prevents invalid entries through defined constraints. The project therefore achieves its main objectives of providing **organized data storage, efficient information retrieval, data integrity, reduced redundancy, and easy management of library operations**. Overall, the developed database provides a strong foundation for a complete Library Management System and can be further extended with additional features in the future.

10.2 Limitations of the Present Work

- No complete graphical user interface or mobile application is implemented.
- Online book reservation functionality is not included.
- Automatic email or SMS notifications are not implemented.
- Online fine payment is not supported.
- Barcode and RFID-based book tracking are not included.
- Fine calculation is not automatically generated based on overdue days.
- User authentication and role-based access are limited.
- Real-time book availability updates are not implemented.
- Automated database backup and recovery are not included.
- The current system focuses mainly on database operations and SQL-based management.

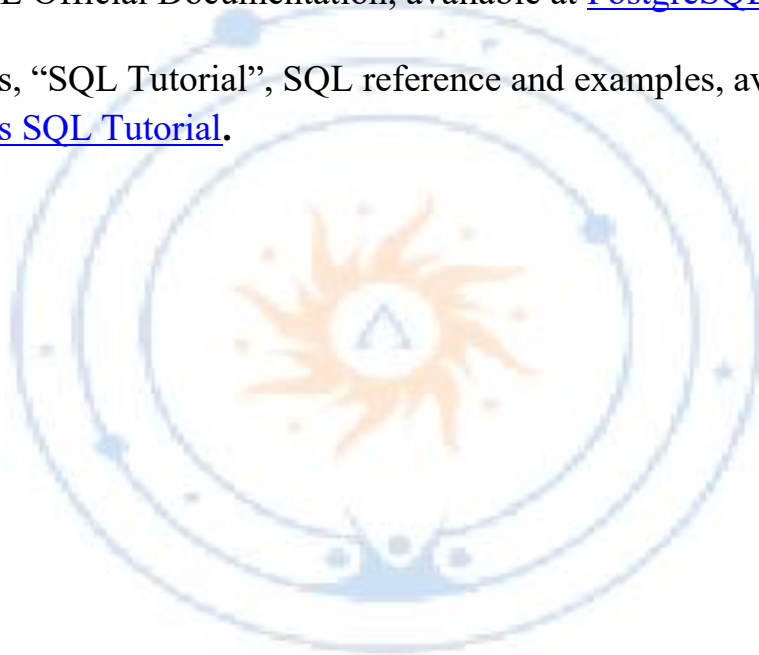
10.3 Future Enhancements

- Develop a **web and mobile application** with a user-friendly interface for librarians and members.
- Add **online book reservation and renewal** facilities.
- Implement **automatic fine calculation** based on overdue days.
- Add **email and SMS notifications** for due dates, returns, and fines.
- Introduce **barcode or RFID scanning** for faster book issue and return.
- Implement **stored procedures, functions, and triggers** to automate database operations.
- Add **indexes** to improve query performance for large amounts of library data.
- Implement **role-based access control** for administrators, librarians, and members.
- Provide **automated database backup and recovery** mechanisms.
- Add **reports and analytics dashboards** for books, members, loans, and fines.
- Improve **security** through stronger authentication, authorization, and data protection.



REFERENCES

1. Abraham Silberschatz, Henry F. Korth and S. Sudarshan, “Database System Concepts”, 8th Edition, McGraw-Hill, ISBN: 978-1260230508.
2. Raghurama Krishnan and Johannes Gehrke, “Database Management Systems”, 4th Edition, McGraw-Hill, ISBN: 978-1260091195.
3. Ramez Elmasri and Shamkant B. Navathe, “Fundamentals of Database Systems”, 7th Edition, Pearson, ISBN: 978-0137504277.
4. PostgreSQL Global Development Group, “PostgreSQL Documentation”, PostgreSQL Official Documentation, available at [PostgreSQL Documentation](#).
5. W3Schools, “SQL Tutorial”, SQL reference and examples, available at [W3Schools SQL Tutorial](#).



APPENDIX A – COMPLETE SQL SCRIPT

A.1 CREATE TABLE Statements

-- Publisher Table

```
CREATE TABLE Publisher (  
    publisher_id INT PRIMARY KEY,  
    publisher_name VARCHAR(100) NOT NULL,  
    country VARCHAR(50),  
    website VARCHAR(150),  
    CONSTRAINT uq_publisher_name UNIQUE (publisher_name),  
    CONSTRAINT uq_publisher_website UNIQUE (website)  
);
```

-- Author Table

```
CREATE TABLE Author (  
    author_id INT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    nationality VARCHAR(50)  
);
```

-- Book Table

```
CREATE TABLE Book (  
    book_id INT PRIMARY KEY,  
    title VARCHAR(200) NOT NULL,  
    isbn VARCHAR(20) NOT NULL,  
    total_copies INT NOT NULL DEFAULT 1,  
    genre VARCHAR(50),  
    publication_year INT,  
    publisher_id INT NOT NULL,  
  
    CONSTRAINT uq_book_isbn UNIQUE (isbn),  
    CONSTRAINT chk_total_copies CHECK (total_copies >= 0),  
    CONSTRAINT chk_publication_year CHECK (publication_year > 0),  
  
    CONSTRAINT fk_book_publisher  
        FOREIGN KEY (publisher_id)  
        REFERENCES Publisher(publisher_id)  
);
```

-- Book_Author Table

```
CREATE TABLE Book_Author (  
    book_id INT NOT NULL,  
    author_id INT NOT NULL,  
  
    PRIMARY KEY (book_id, author_id),  
  
    CONSTRAINT fk_book_author_book  
        FOREIGN KEY (book_id)  
        REFERENCES Book(book_id)
```

```

ON DELETE CASCADE,

CONSTRAINT fk_book_author_author
  FOREIGN KEY (author_id)
  REFERENCES Author(author_id)
  ON DELETE CASCADE
);

-- Member Table
CREATE TABLE Member (
  member_id INT PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) NOT NULL,
  membership_type VARCHAR(30) NOT NULL,
  membership_date DATE NOT NULL,
  membership_expiry DATE,

  CONSTRAINT uq_member_email UNIQUE (email)
);

-- Member_Phone Table
CREATE TABLE Member_Phone (
  member_id INT NOT NULL,
  phone_number VARCHAR(15) NOT NULL,

  PRIMARY KEY (member_id, phone_number),

  CONSTRAINT fk_member_phone_member
    FOREIGN KEY (member_id)
    REFERENCES Member(member_id)
    ON DELETE CASCADE
);

-- Loan Table
CREATE TABLE Loan (
  loan_id INT PRIMARY KEY,
  book_id INT NOT NULL,
  member_id INT NOT NULL,
  loan_date DATE NOT NULL,
  due_date DATE NOT NULL,
  return_date DATE,

  CONSTRAINT fk_loan_book
    FOREIGN KEY (book_id)
    REFERENCES Book(book_id),

  CONSTRAINT fk_loan_member
    FOREIGN KEY (member_id)
    REFERENCES Member(member_id),

  CONSTRAINT chk_due_date
    CHECK (due_date >= loan_date),

```

```

CONSTRAINT chk_return_date
CHECK (
    return_date IS NULL
    OR return_date >= loan_date
)
);

-- Fine Table
CREATE TABLE Fine (
    fine_id INT PRIMARY KEY,
    loan_id INT NOT NULL,
    member_id INT NOT NULL,
    fine_amount DECIMAL(10,2) NOT NULL DEFAULT 0,
    fine_date DATE NOT NULL,
    payment_status VARCHAR(20) NOT NULL DEFAULT 'Unpaid',

    CONSTRAINT uq_fine_loan UNIQUE (loan_id),

    CONSTRAINT fk_fine_loan
        FOREIGN KEY (loan_id)
        REFERENCES Loan(loan_id),

    CONSTRAINT fk_fine_member
        FOREIGN KEY (member_id)
        REFERENCES Member(member_id),

    CONSTRAINT chk_fine_amount
        CHECK (fine_amount >= 0),

    CONSTRAINT chk_payment_status
        CHECK (payment_status IN ('Paid', 'Unpaid'))
);

```

A.2 INSERT Statements

-- Publisher Data

INSERT INTO Publisher VALUES

```

(1, 'Pearson', 'USA', 'www.pearson.com'),
(2, 'O'Reilly Media', 'USA', 'www.oreilly.com'),
(3, 'McGraw Hill', 'USA', 'www.mheducation.com');

```

-- Author Data

INSERT INTO Author VALUES

```

(1, 'Abraham', 'Silberschatz', 'USA'),
(2, 'Robert', 'Martin', 'USA'),
(3, 'Andrew', 'Tanenbaum', 'Netherlands'),
(4, 'Herbert', 'Schildt', 'USA'),
(5, 'Thomas', 'Cormen', 'USA');

```

-- Book Data

INSERT INTO Book VALUES

```

(1, 'Database System Concepts', 'ISBN001', 10, 'Database', 2022, 1),
(2, 'Clean Code', 'ISBN002', 5, 'Programming', 2021, 2),
(3, 'Computer Networks', 'ISBN003', 4, 'Networking', 2020, 1),

```

```
(4, 'Operating System Concepts', 'ISBN004', 7, 'Operating Systems', 2022, 3),
(5, 'Python Programming', 'ISBN005', 6, 'Programming', 2023, 2),
(6, 'Data Structures and Algorithms', 'ISBN006', 8, 'Computer Science', 2021, 3);
```

-- Book_Author Data

```
INSERT INTO Book_Author VALUES
```

```
(1, 1),
(2, 2),
(3, 3),
(4, 3),
(5, 4),
(6, 5);
```

-- Member Data

```
INSERT INTO Member VALUES
```

```
(1, 'Priya', 'Sharma', 'priya@gmail.com', 'Student',
'2026-01-10', '2027-01-10'),
(2, 'Ravi', 'Kumar', 'ravi@gmail.com', 'Student',
'2026-02-15', '2027-02-15'),
(3, 'Arjun', 'Reddy', 'arjun@gmail.com', 'Faculty',
'2026-03-01', '2027-03-01'),
(4, 'Sneha', 'Patel', 'sneha@gmail.com', 'Student',
'2026-03-20', '2027-03-20');
```

-- Member Phone Data

```
INSERT INTO Member_Phone VALUES
```

```
(1, '9876543210'),
(2, '9876543211'),
(3, '9876543212'),
(4, '9876543213');
```

-- Loan Data

```
INSERT INTO Loan VALUES
```

```
(1, 1, 1, '2026-09-01', '2026-09-08', '2026-09-07'),
(2, 2, 2, '2026-09-10', '2026-09-17', NULL),
(3, 4, 3, '2026-09-12', '2026-09-19', '2026-09-18'),
(4, 5, 4, '2026-09-15', '2026-09-22', NULL),
(5, 6, 1, '2026-09-16', '2026-09-23', '2026-09-22'),
(6, 3, 2, '2026-09-18', '2026-09-25', NULL);
```

-- Fine Data

```
INSERT INTO Fine VALUES
```

```
(1, 1, 1, 20.00, '2026-09-07', 'Paid'),
(2, 3, 3, 30.00, '2026-09-18', 'Paid'),
(3, 5, 1, 10.00, '2026-09-22', 'Unpaid');
```

A.3 Basic DQL Queries

-- Q1: Display all books

```
SELECT book_id, title, isbn, publisher_id
FROM Book
ORDER BY book_id;
```

-- Q2: Books having more than 5 copies

```
SELECT book_id, title, total_copies
```

```
FROM Book
WHERE total_copies > 5
ORDER BY title;
```

```
-- Q3: Display currently active loans
SELECT loan_id, book_id, member_id, loan_date, due_date
FROM Loan
WHERE return_date IS NULL
ORDER BY loan_id;
```

A.4 Join Queries

```
-- Q4: Natural Join
SELECT book_id, title, publisher_name
FROM Book
NATURAL JOIN Publisher;
```

```
-- Q5: Equi Join
SELECT b.book_id, b.title, p.publisher_name
FROM Book b, Publisher p
WHERE b.publisher_id = p.publisher_id;
```

```
-- Q6: Inner Join
SELECT b.book_id, b.title, p.publisher_name
FROM Book b
INNER JOIN Publisher p
ON b.publisher_id = p.publisher_id;
```

```
-- Q7: Left Outer Join
SELECT b.book_id, b.title, p.publisher_name
FROM Book b
LEFT JOIN Publisher p
ON b.publisher_id = p.publisher_id;
```

```
-- Q8: Right Outer Join
SELECT b.book_id, b.title, p.publisher_name
FROM Book b
RIGHT JOIN Publisher p
ON b.publisher_id = p.publisher_id;
```

```
-- Q9: Full Outer Join
SELECT b.book_id, b.title, p.publisher_name
FROM Book b
FULL OUTER JOIN Publisher p
ON b.publisher_id = p.publisher_id;
```

A.5 Aggregate and GROUP BY Queries

```
-- Q10: Count total books
SELECT COUNT(*) AS total_books
FROM Book;
```

```
-- Q11: Total copies
SELECT SUM(total_copies) AS total_copies
FROM Book;
```

```
-- Q12: Average number of copies
SELECT AVG(total_copies) AS average_copies
FROM Book;

-- Q13: Maximum copies
SELECT MAX(total_copies) AS maximum_copies
FROM Book;

-- Q14: Minimum copies
SELECT MIN(total_copies) AS minimum_copies
FROM Book;

-- Q15: Number of books by publisher
SELECT publisher_id, COUNT(*) AS book_count
FROM Book
GROUP BY publisher_id
ORDER BY publisher_id;

-- Q16: Total copies by publisher
SELECT publisher_id, SUM(total_copies) AS total_copies
FROM Book
GROUP BY publisher_id
ORDER BY publisher_id;

-- Q17: Publishers having more than one book
SELECT publisher_id, COUNT(*) AS book_count
FROM Book
GROUP BY publisher_id
HAVING COUNT(*) > 1
ORDER BY publisher_id;
```

A.6 Nested and Correlated Subqueries

```
-- Q18: Books having copies above the average
SELECT book_id, title, total_copies
FROM Book
WHERE total_copies > (
    SELECT AVG(total_copies)
    FROM Book
)
ORDER BY book_id;

-- Q19: Members who have borrowed at least one book
SELECT m.member_id, m.first_name, m.last_name
FROM Member m
WHERE EXISTS (
    SELECT 1
    FROM Loan l
    WHERE l.member_id = m.member_id
)
ORDER BY m.member_id;
```

A.7 Views

```
-- View 1: Book and Publisher Details
CREATE VIEW v_book_publisher AS
SELECT b.book_id,
```

```
b.title,  
b.isbn,  
p.publisher_name,  
b.genre  
FROM Book b  
JOIN Publisher p  
ON b.publisher_id = p.publisher_id;
```

```
-- View 2: Book Summary by Genre  
CREATE VIEW v_book_summary AS  
SELECT genre,  
       COUNT(book_id) AS total_books  
FROM Book  
GROUP BY genre;
```

A.8 Queries on Views

```
-- Q20: Display book-publisher report  
SELECT book_id, title, publisher_name, genre  
FROM v_book_publisher  
ORDER BY book_id;
```

```
-- Q21: Display genre-wise book summary  
SELECT genre, total_books  
FROM v_book_summary  
ORDER BY total_books DESC;
```

A.9 Set Operations

```
-- Q22: UNION  
SELECT genre  
FROM Book  
WHERE genre = 'Programming'
```

UNION

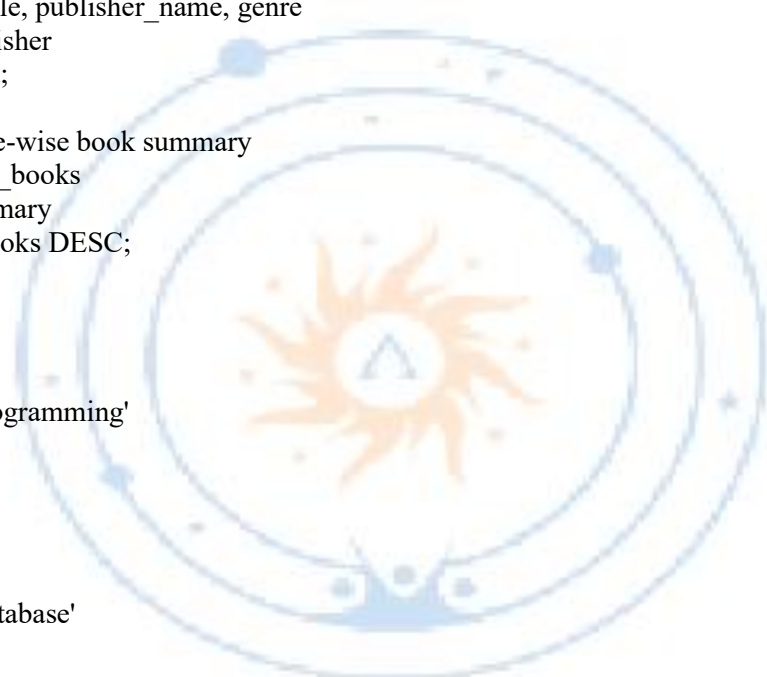
```
SELECT genre  
FROM Book  
WHERE genre = 'Database'  
ORDER BY genre;
```

```
-- Q23: INTERSECT  
SELECT genre  
FROM Book  
WHERE total_copies >= 5
```

INTERSECT

```
SELECT genre  
FROM Book  
WHERE publication_year >= 2020  
ORDER BY genre;
```

```
-- Q24: EXCEPT  
SELECT genre  
FROM Book  
WHERE total_copies >= 5
```



EXCEPT

```
SELECT genre
FROM Book
WHERE publication_year >= 2020
ORDER BY genre;
```

A.10 DML Operations

```
-- INSERT
INSERT INTO Book
VALUES
(7, 'SQL Fundamentals', 'ISBN007', 4,
'Database', 2024, 1);
```

```
-- UPDATE
UPDATE Book
SET total_copies = 5
WHERE book_id = 7;
```

```
-- DELETE
DELETE FROM Book
WHERE book_id = 7;
```

A.11 DCL Operations

```
-- Create user
CREATE USER exam_cell
WITH PASSWORD 'exam@123';

-- Grant permission
GRANT SELECT ON Book TO exam_cell;
```

```
-- Grant permission on Loan
GRANT SELECT ON Loan TO exam_cell;
```

```
-- Revoke permission
REVOKE SELECT ON Book FROM exam_cell;
```

A.12 TCL Operations

```
BEGIN;
```

```
UPDATE Fine
SET fine_amount = 50
WHERE fine_id = 1;
```

```
SAVEPOINT after_first_update;
```

```
UPDATE Fine
SET fine_amount = 100
WHERE fine_id = 2;
```

```
ROLLBACK TO SAVEPOINT after_first_update;
```

```
COMMIT;
```

A.13 Verification Query

```
SELECT fine_id, fine_amount
```

```
FROM Fine  
WHERE fine_id IN (1, 2)  
ORDER BY fine_id;
```

